CA2 PROJECT

Cloud-Native Monitoring & Autoscaling

FitMotion Web Application on Kubernetes

Student: Pranav Kumar
Reg No: 12209660 **Roll:** 42 **Section:** 4OM05

DevOps — Kubernetes · Docker · Prometheus · Grafana · HPA

Project Overview

Containerize

Dockerized the FitMotion React app using a multi-stage Dockerfile (build → Nginx serve)

Deploy on K8s

Ran on Minikube (local Kubernetes) with deployment + service manifests

Monitor

Full observability via Kube-Prometheus-Stack (Prometheus + Grafana) installed with Helm

Auto-Scale

HPA automatically scales pods from 2 → 10 when CPU exceeds the threshold

System Architecture

FitMotion (React + Vite)

Frontend Web App

Kubernetes / Minikube

Container Orchestration Platform

Prometheus + Grafana

Metrics Collection & Visualisation

HPA – Horizontal Pod Autoscaler

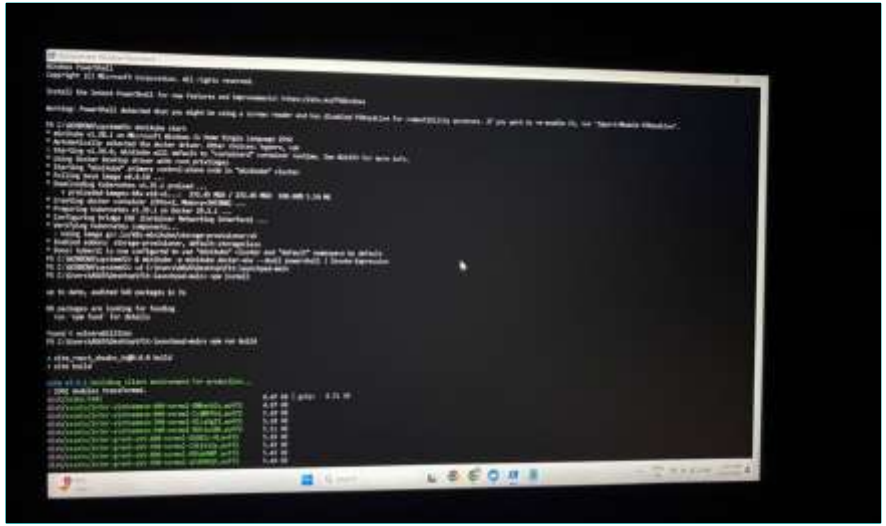
Auto-scales pods based on CPU metrics

Key Tools Used

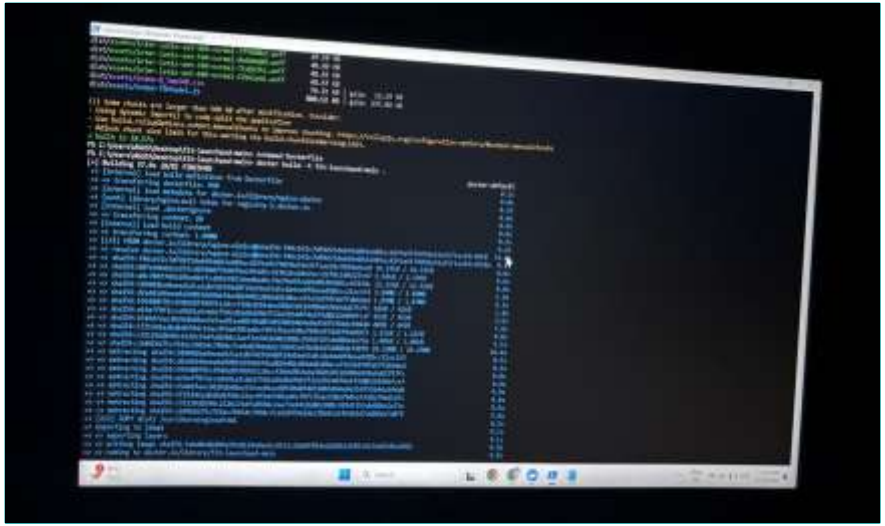
- Docker
- Minikube
- Helm
- Prometheus
- Grafana
- HPA

Step 1 — Environment Setup & App Build

Minikube cluster started on Windows 11 + FitMotion React app built with npm → Vite



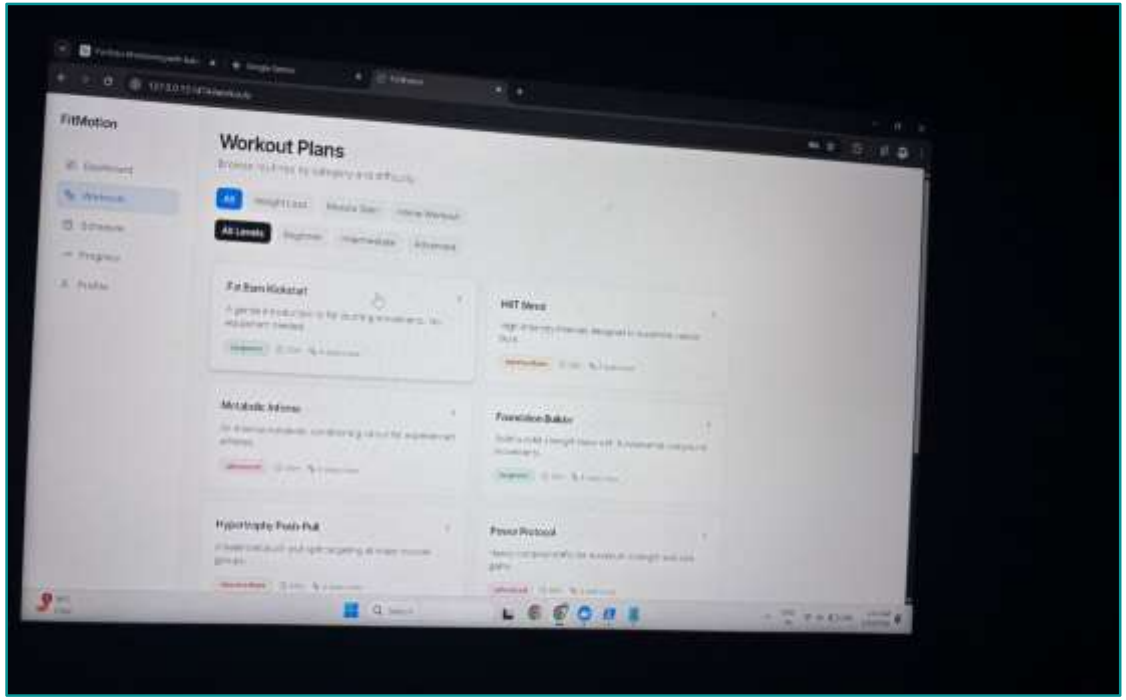
Minikube start + npm install + build



Docker image build (multi-stage → Nginx)

Commands: minikube start | npm run build | docker build -t fit-launchpad-main .

Step 2 — Application Deployment & Verification



FitMotion app running at localhost via port-forward

01 kubectl apply -f deployment.yaml

02 kubectl apply -f service.yaml

03 kubectl port-forward svc/fit-launchpad-service 1414:80

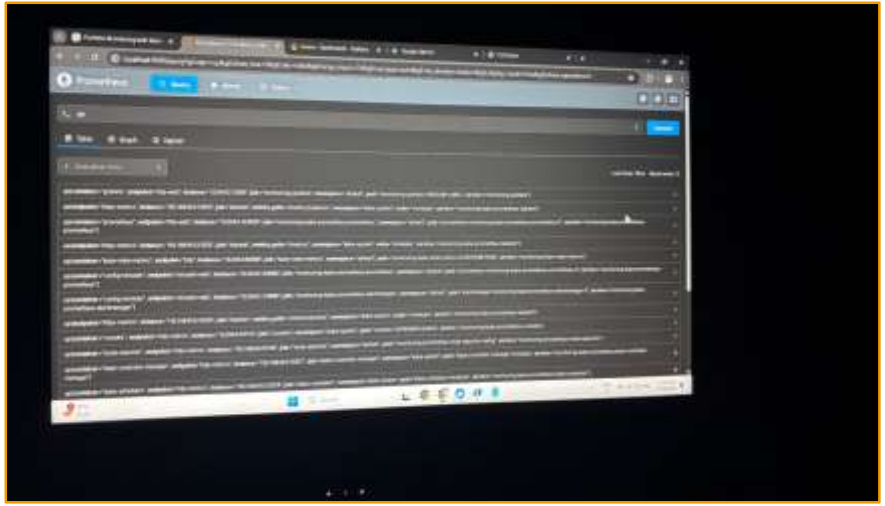
04 App verified at http://127.0.0.1:1414

App Deployed Successfully!

Step 3 — Monitoring Stack: Prometheus & Grafana

Kube-Prometheus-Stack deployed via Helm. Prometheus scrapes all cluster components; Grafana visualises metrics.

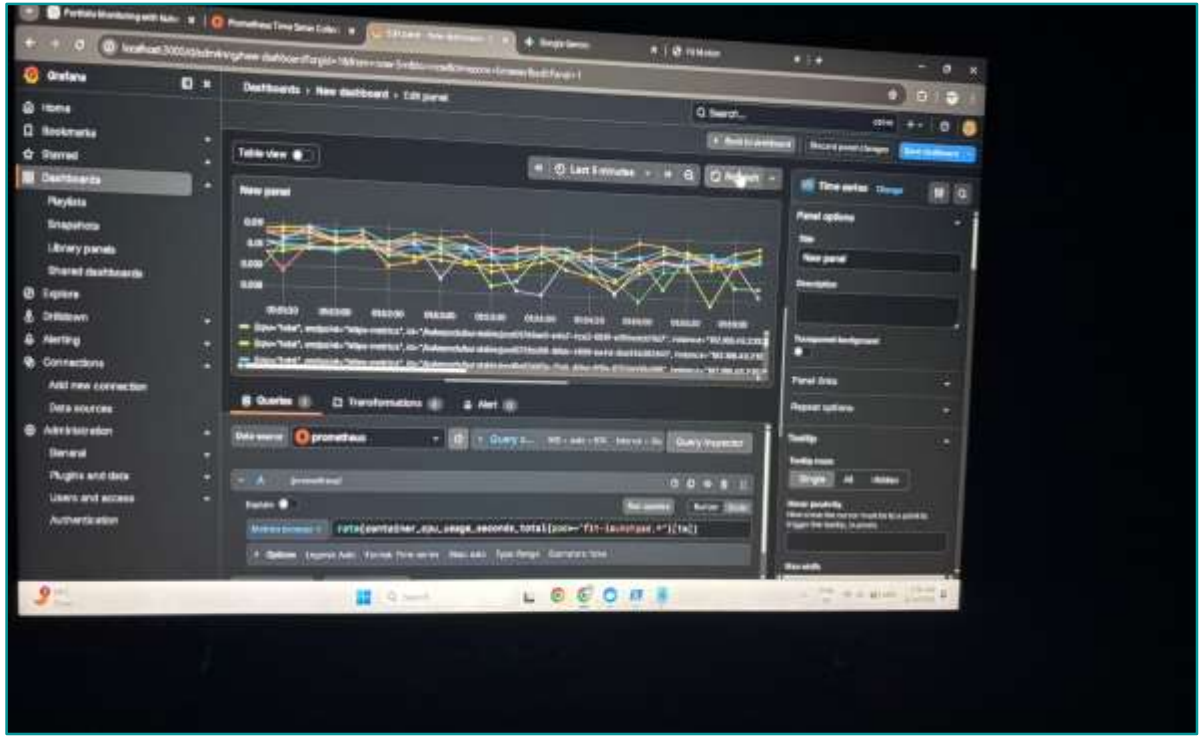
Helm install – STATUS: deployed ✓



Prometheus – 17 scrape targets all UP ✓

Helm command: helm install monitoring prometheus-community/kube-prometheus-stack

Step 4 — Grafana Dashboard (Custom PromQL)



Dashboard Details

Data Source:

Prometheus

Query (PromQL):

rate(container_cpu_usage_seconds_total{pod=~"fit-launchpad.*"}[1m])

Refresh:

Every 15s

Chart type:

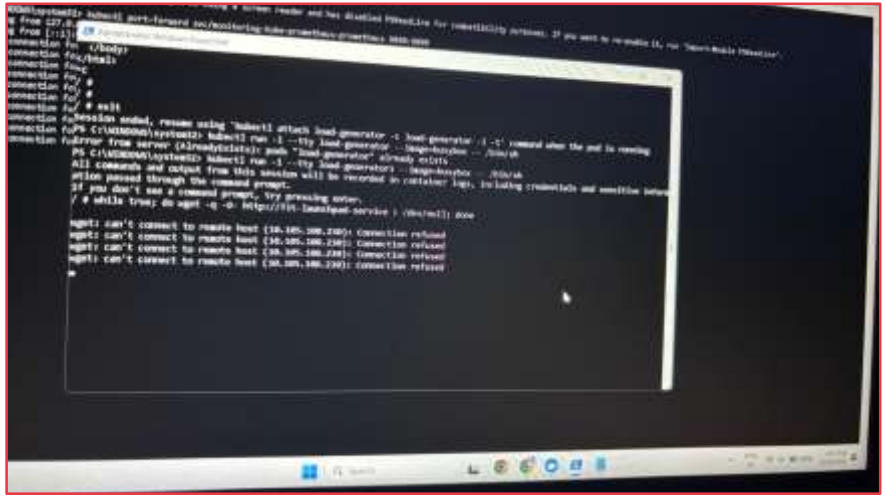
Time Series

Purpose:

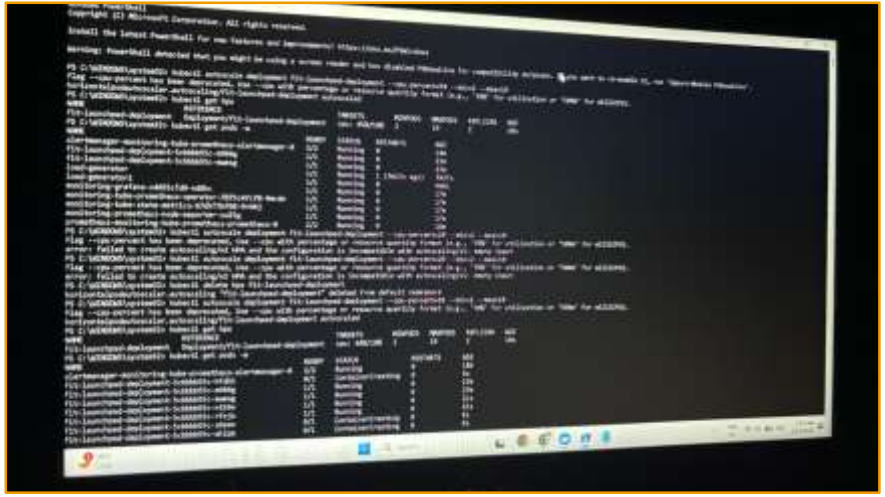
Live CPU tracking per pod

Step 5 — Load Testing & Troubleshooting

A busybox load-generator pod sent infinite HTTP requests to stress the app and trigger HPA.



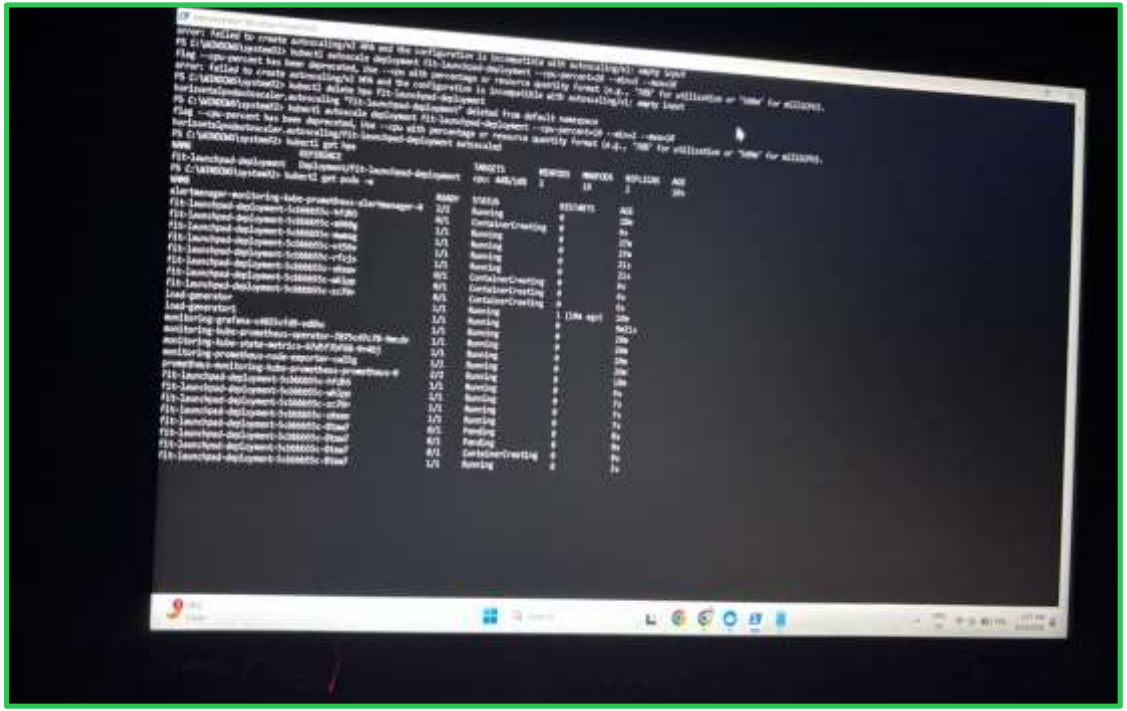
⚠ Connection refused errors during load test



HPA API version errors → fixed with autoscaling/v2

Issue 1: Service ClusterIP not reachable from busybox pod — fixed service selector. **Issue 2:** --cpu-percent deprecated → replaced with autoscaling/v2 format.

Step 6 — Autoscaling in Action (HPA)



Pods scaling up from 2 → 10 in real-time

2	Min Replicas
10	Max Replicas
10%	CPU Trigger
44/10 %	Actual / Target CPU

HPA working perfectly ✓

CONCLUSION & OUTCOMES

What We Achieved

- ✓ Successfully containerized FitMotion with Docker & Nginx
- ✓ Deployed on Kubernetes (Minikube) with zero manual intervention
- ✓ Full observability via Prometheus (17 targets) + Grafana dashboard
- ✓ HPA auto-scaled deployment from 2 → 10 pods under load
- ✓ Resolved real-world issues: network config & API versioning
- ✓ Zero downtime during high-traffic simulation

Key Learnings

- DevOps Pipeline
- Cloud-Native Design
- Real-time Metrics
- Self-Healing Infra